

# Quantum Routing via Beam Search

We solved QSITE’s computational challenge via a beam search algorithm. For the first five challenges, we can exhaustively bruteforce every potential solution to determine **our score is optimal**. We weren’t able to bruteforce `dense_random` due to its extreme complexity, but after searching through several billion solutions, we found numerous at 34.5 and none below. Thus it’s assumed this is also (at least close to) optimal.

| benchmark      | depth | swaps | score       |
|----------------|-------|-------|-------------|
| ghz_star       | 9     | 2     | 6.5         |
| chain_trotter  | 9     | 0     | 4.5         |
| ladder_trotter | 7     | 3     | 6.5         |
| qaoa_random    | 11    | 6     | 11.5        |
| dense_random   | 23    | 2     | 34.5        |
| vqe_layers     | 6     | 0     | 3.0         |
| <b>total</b>   |       |       | <b>66.5</b> |

## Overview

The core of our solution is a beam search algorithm, which looks something like:

```
def beam_search(program, graph):
    states = [State()]
    for i, gate in enumerate(program):
        upcoming = program[i + 1:]
        candidates = proactive_swap(run_gate(states, gate), upcoming)
        states = keep_best(candidates, upcoming)
    return states[0]
```

That is, we maintain a list of current states, the first state being the empty circuit. Then, for each gate we have to route, 1. Run the gate on all current states. 2. Proactively swap qubits around to prepare for future gates. 3. Keep only the best  $B = 2048$  candidates for the next iteration. Then return the single best circuit at the end.

Best here is defined as `score(program) + sum(distances[a][b] - 1 for a, b in upcoming_gates[:20]) * 0.5`, meaning the score plus a heuristic term penalizing far-away upcoming 2Q operations. This term roughly equals the number of SWAPs we’d have to make in the future.

## Verification